15-442/15-642: Machine Learning Systems

Blackwell GPU and TIRx

Spring 2026

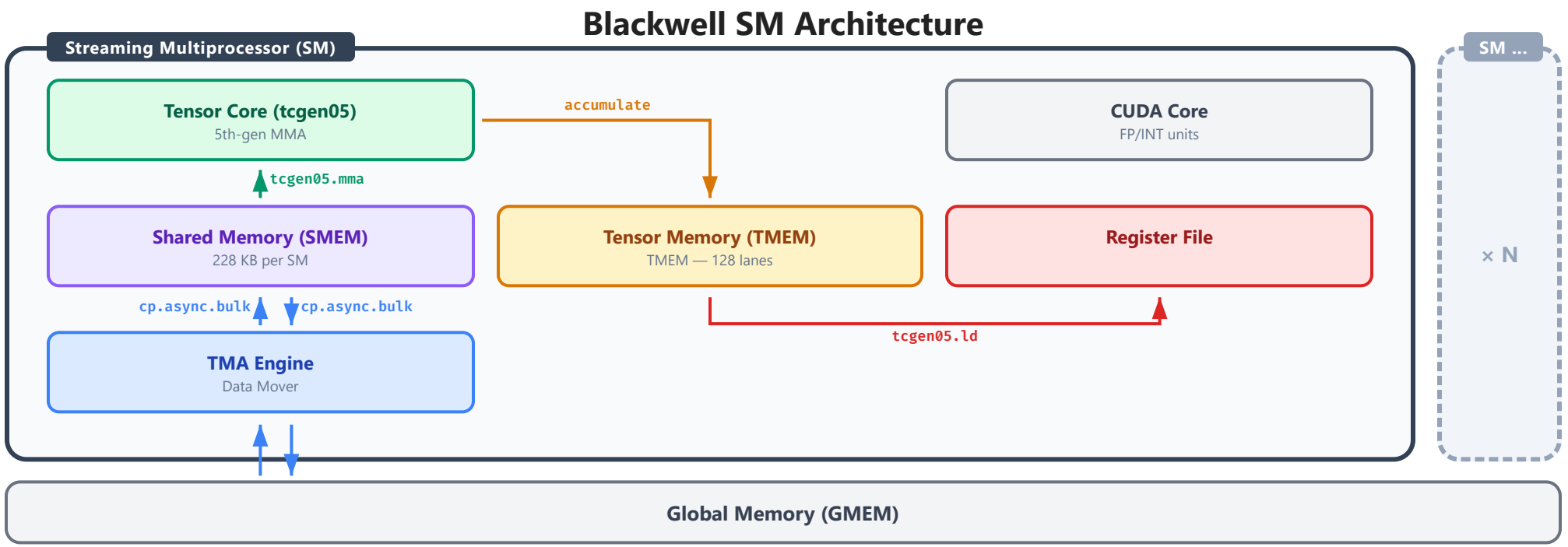
Tianqi Chen and Zhihao Jia

Carnegie Mellon University

Outline

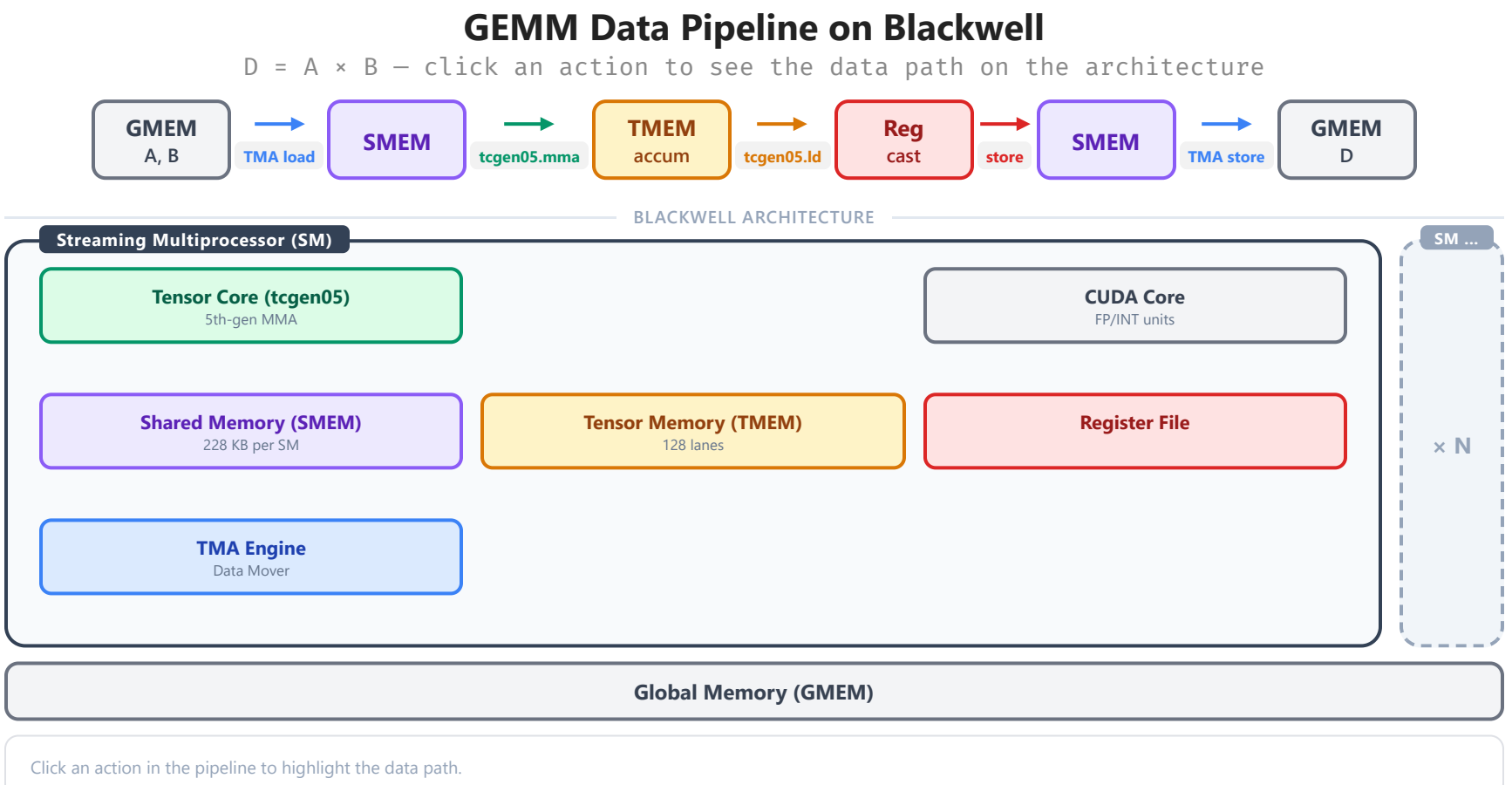
Outline

Blackwell SM Architecture

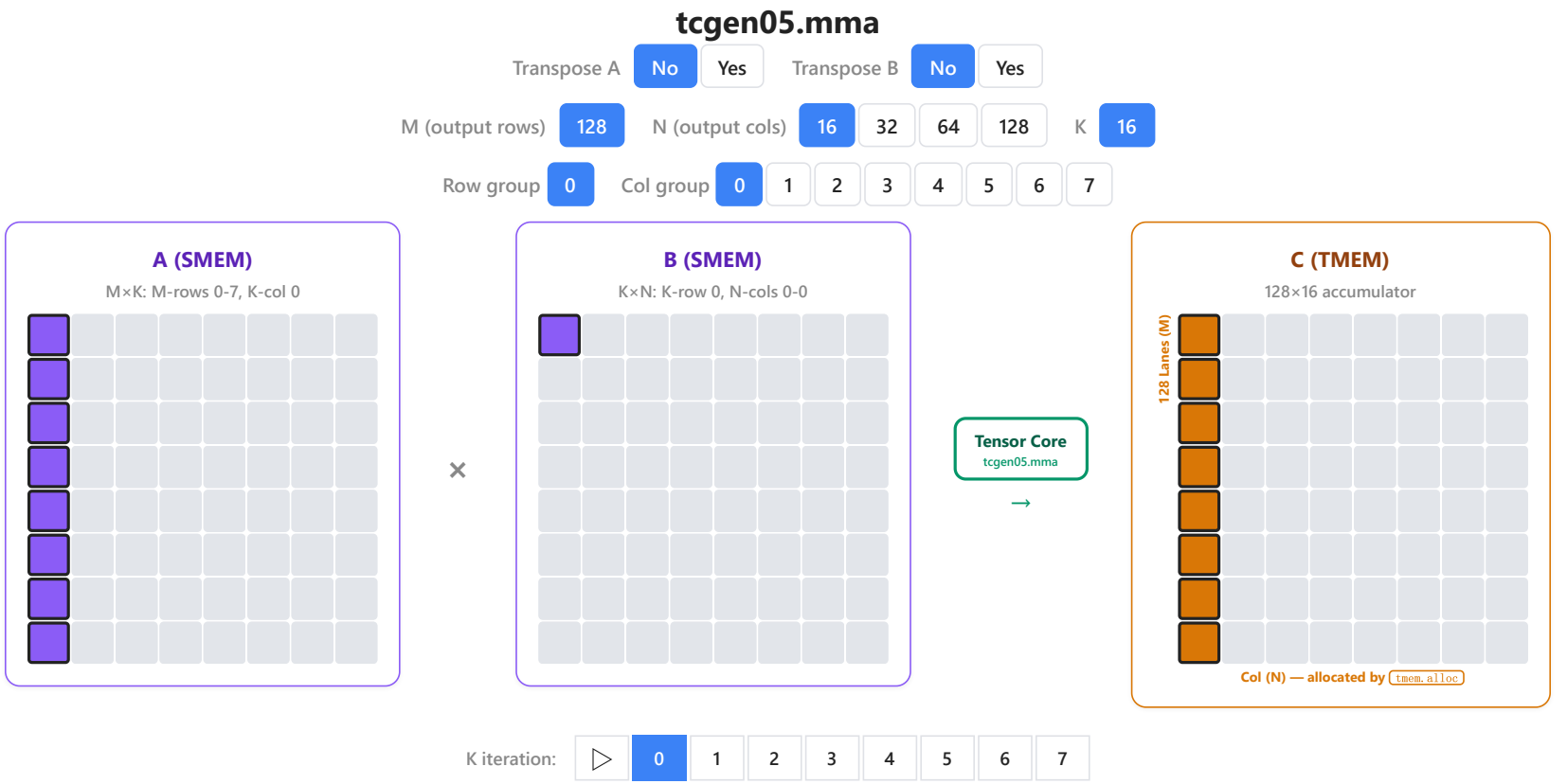


Click a hardware unit to see details. Solid arrows = load path, dashed arrows = store path.

Example GEMM Data Pipeline on Blackwell



tcgen05 & Tensor Memory

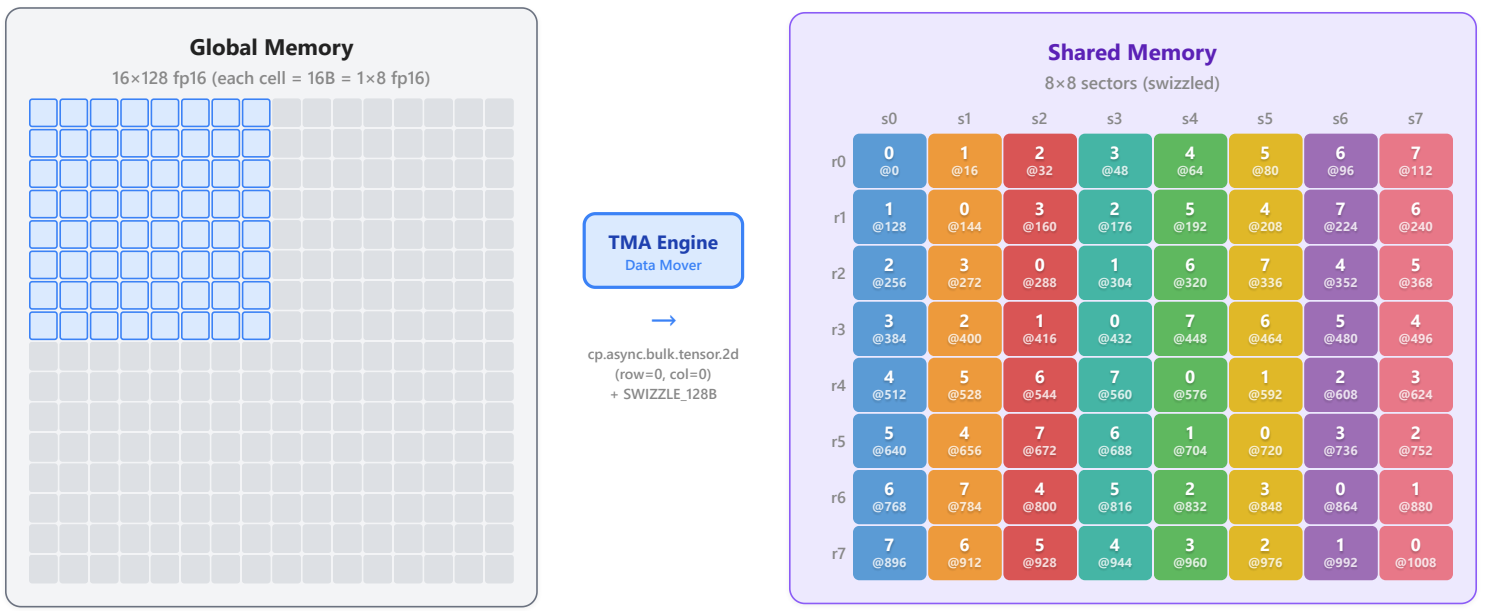


TMA: Tensor Memory Accelerator

TMA: Global to Shared Memory

Hardware 2D copy with optional swizzle — one instruction, no threads

Swizzle: None | 128B — Row offset: 0 | 8 — Col offset: 0 | 64



TensorMap: `dtype = f16, globalDim = [128, 16], globalStrides = [256B], boxDim = [64, 8], swizzle = CU_TENSOR_MAP_SWIZZLE_128B`

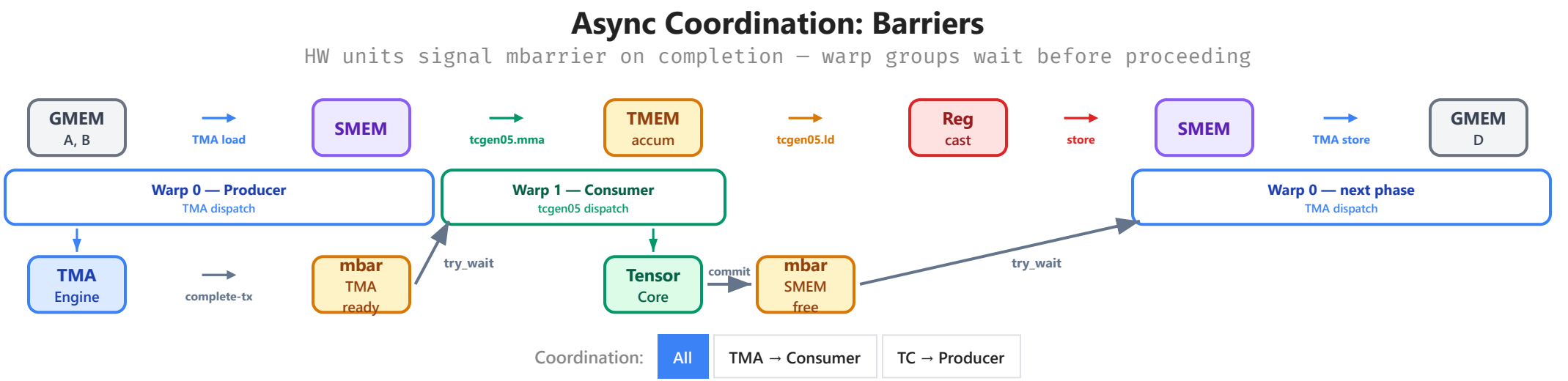
Instruction: `cp.async.bulk.tensor.2d.shared::cta.global.mbarrier::complete_tx::bytes [smem], [tensormap, {0, 0}], [mbar]`

Note: TensorMap and PTX follows column-major per official spec — globalDim, globalStrides, boxDim, and copy instruction coordinates are reversed from row-major order.

sector 0 | sector 1 | sector 2 | sector 3 | sector 4 | sector 5 | sector 6 | sector 7 | Cell value = logical col index | @N = smem byte offset

- **Single-thread dispatch** (non-blocking)
- **Automatic swizzling** — data arrives bank-conflict-free
- **Bidirectional:** load (GMEM→SMEM) and store (SMEM→GMEM)

Async Coordination: Barriers



Warp 0 (producer) and Warp 1 (consumer) run in parallel, coordinated by mbarriers

- **mbarrier (mbar)** — used to coordinate asynchronous operations across threads and hardware units
- Different warps run in parallel on different stages of operations

MBarrier: Data Structure & APIs

MBarrier: Data Structure & APIs

MBarrier Object (64-bit, in shared memory)

`phase`	Current phase (0 or 1). Flips automatically on completion.
`pending_count`	Remaining arrivals needed before phase completes.
`expected_count`	Total arrivals expected per phase (set by `init`).
`tx-count`	Pending async bytes. Increased by `arrive.expect_tx`, decreased by HW on transfer completion.

Phase Completion Condition

```
pending_count == 0
&&
tx-count ≤ 0
```

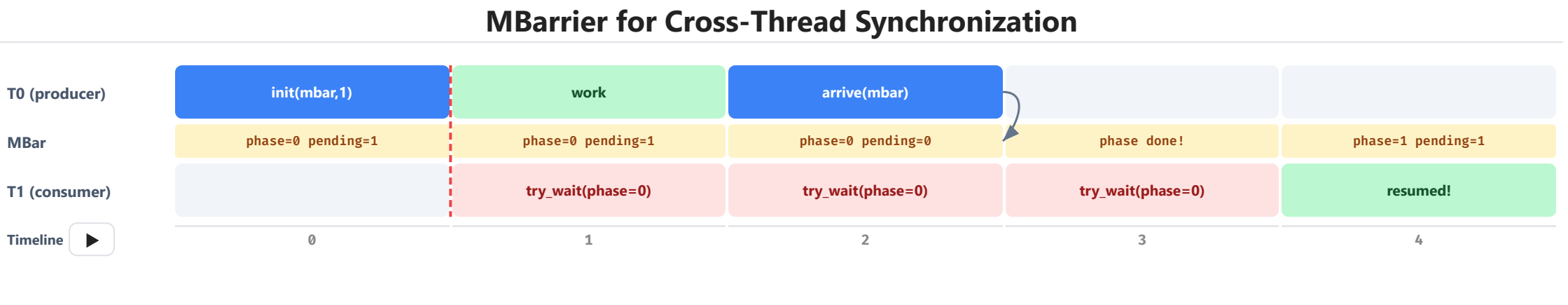
A phase completes when **all expected threads have arrived** AND **all expected async bytes have been transferred**. Upon completion, the barrier auto-resets: phase flips, pending_count resets to expected_count, tx-count resets to 0.

Core APIs

INSTRUCTION	ROLE	EFFECT ON MBARRIER
`mbarrier.init` `setup`	Initialize barrier with expected arrival count	`phase=0, pending=count, tx=0`
`mbarrier.arrive` `producer`	Thread signals arrival at the barrier	`pending_count -= 1`
`mbarrier.arrive.expect_tx` `producer`	Arrive + declare expected async transfer bytes	`pending -= 1, tx_count += txCount`
`tcgen05.commit` `producer`	Tensor core signals MMA completion	`arrive::one on mbarrier`
`mbarrier.try_wait` `consumer`	Wait (suspend thread) until phase completes	`blocks until phase done`

Three arrival patterns: (1) `mbarrier.arrive` — thread directly decrements pending_count. (2) `arrive.expect_tx` — thread arrives + declares bytes; TMA engine auto-decrements tx_count when transfer finishes. (3) `tcgen05.commit` — tensor core auto-arrives when MMA completes.

MBarrier for Cross-Thread Synchronization



Setup (T0)

`mbarrier.init(mbar, 1)` — setup once across all runs, explicit sync after setup.

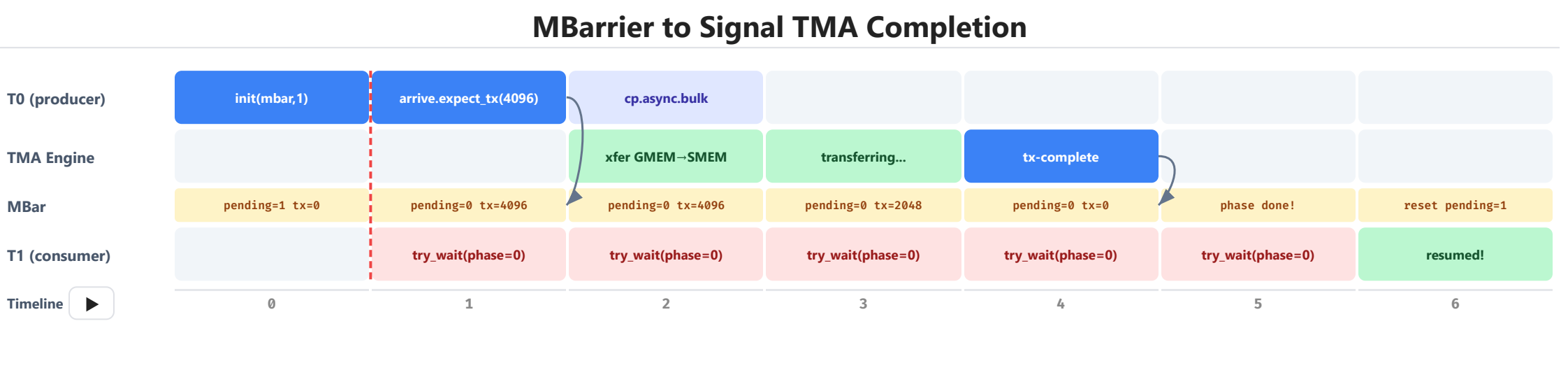
Producer (T0)

`mbarrier.arrive(mbar)` decrements pending_count. When it hits 0, phase completes.

Consumer (T1)

`mbarrier.try_wait(mbar, phase)` suspends T1 until phase completes.

MBarrier to Signal TMA Completion



Producer (T0)

`arrive.expect_tx(4096)` decrements pending to 0 and sets tx_count. `cp.async.bulk` triggers TMA transfer.

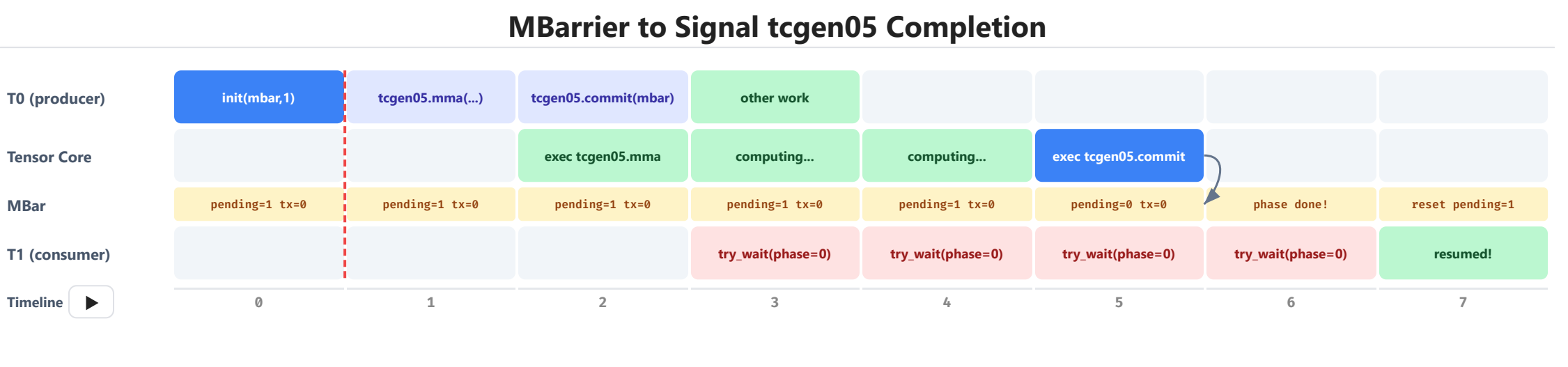
TMA Engine

Executes `cp.async.bulk` dispatched from T0. Auto-decrements tx_count on completion.

Consumer (T1)

`try_wait` suspends until pending==0 AND tx==0. HW auto-decrements tx_count when transfer completes.

MBarrier to Signal tcgen05 Completion



Working / active · Signaling operation · Dispatch to TensorCore · Blocked / waiting · Barrier state · Idle · Signaling to change mbar state · Sync threads after init

Producer (T0)

`tcgen05.mma` issues async matrix multiply. `tcgen05.commit(mbar)` tells barrier to track it; HW will `arrive::one` when done.

Tensor Core

Executes `tcgen05.mma` and `tcgen05.commit` queued from T0.

Consumer (T1)

`mbarrier.try_wait(mbar, phase)` suspends T1 until phase completes. Result available in TMEM.

Phase Tracking

Phase Tracking

Barriers reuse across iterations by flipping phase 0 ↔ 1

▶ Animate

How It Works

Each barrier has a **phase bit** (0 or 1). After all expected arrivals, the barrier flips its phase and resets — ready for the next iteration. No reinitialization needed.

1. Producer arrives at barrier (phase 0)
2. Consumer calls `try_wait(phase=0)` — unblocks
3. Barrier flips to phase 1, `phase ^= 1`

Why Phase Tracking?

In a pipelined kernel, barriers are reused every iteration of the K-loop. Phase tracking solves the problem of **"did this barrier fire for iteration *i* or iteration *i-1*?"**

```
phase_tma = 0
for k in range(K):
  try_wait(tma_bar, phase_tma)
  phase_tma ^= 1  # flip
```

Outline

What is TIRx: Native Level Access

TIRx provides native-level (CUDA C/PTX) access to hardware

```python
@Tx.prim_func(tirx=True)
def raw_CUDA_ptx_tcgen05(A: Tx.Buffer((128 * 64), "float16"), B: Tx.Buffer((128 * 64), "float16"), C: Tx.Buffer((128, 128), "float32")):

    with Tx.kernel():
        bx = Tx.cta_id([1], parent="kernel")
        warp_id = Tx.warp_id([4], parent="cta")
        lane_id = Tx.thread_id([32], parent="warp")
        tx = Tx.thread_id([128], parent="cta")
        warp_id_in_cta: Tx.let[Tx.int32] = Tx.tvm_warp_shuffle(0xFFFFFFFF, tx // 32, 0, 32, 32)

        dyn = Tx.alloc_buffer((33792,), "uint8", scope="shared.dyn")
        tmem_addr = Tx.decl_buffer((1,), "uint32", data=dyn.data, scope="shared")
        A_smem = Tx.decl_buffer((8192,), "float16", data=dyn.data, elem_offset=256, scope="shared")
        B_smem = Tx.decl_buffer((8192,), "float16", data=dyn.data, elem_offset=8448, scope="shared")
        bar = Tx.decl_buffer((1,), "uint64", data=dyn.data, elem_offset=8, scope="shared")
        reg = Tx.alloc_local((128,))
        descA = Tx.alloc_local((1,), "uint64")
        descB = Tx.alloc_local((1,), "uint64")
        descI = Tx.alloc_local((1,), "uint32")
        phase = Tx.alloc_local((1,), "int32")

        if 0 <= warp_id_in_cta and warp_id_in_cta < 1:
            Tx.ptx.tcgen05.alloc(Tx.address_of(tmem_addr[0]), 512, 1)
        Tx.cuda.cta_sync()
        for i in range(128):
            reg[i] = Tx.float32(0.0)
        if tx == 0:
            for v_2, v_3 in T.grid(128, 64):
                idx = v_2 * 8 + v_3 // 8
                A_smem[((idx ^ ((idx & 56) >> 3)) << 3) + v_3 % 8 + 256] = A[v_2 * 64 + v_3]
        Tx.cuda.cta_sync()
        if tx == 0:
            for v_2, v_3 in T.grid(128, 64):
                idx = v_2 * 8 + v_3 // 8
                B_smem[((idx ^ ((idx & 56) >> 3)) << 3) + v_3 % 8 + 8448] = B[v_2 * 64 + v_3]
        Tx.cuda.cta_sync()

        Tx.ptx.fence.proxy_async("shared::cta")
        Tx.cuda.cta_sync()
        phase[0] = 0
```

TIRX → CUDA C/PTX MAPPING

← Click a highlighted code block to inspect its CUDA C/PTX counterpart. Together, these elements make TIRx DSL expressively equivalent for representing CUDA C/PTX kernels.

Native Level Access

Q: Why didn't pre-FlashAttention ML compilers generate FlashAttention-style attention kernels?

AI workloads and hardware outpace DSL adaptation and maintenance.

- **TIRx is our ongoing effort on ML compiler DSL for agentic and frontier programming R&D needs (e.g. megakernels).**
- We already have many DSLs (`TileLang`[1], `Triton`[2], `ThunderKittens`[3], `TVM`[4], ...), and history has seen many more come and go.
- In practice, the strongest kernels are still often written in `CUDA`, or built with CUDA-native libraries like `CUTLASS`.
- SOTA optimizations and GPU features evolve quickly from generation to generation.
- We are now also seeing native-first DSLs rising due to these same reasons (`Gluon`[5], `CuteDSL`[6]).

One recurring bottleneck:

1. User (**Human** / **Agent**) cannot express the best kernel optimizations in DSL.
2. The compiler passes of DSL fail to utilize the latest optimizations/hardware features automatically when DSLs cannot express them.
3. Users (**Human** / **Agent**) have to open the box and dig into compiler passes to inject optimizations, which can potentially introduce new failures.

> **Design takeaway:** native-level access should always be part of the programming model for agents and human to get the state of art performance.

DSL references: [1] TileLang (tile-ai/tilelang), [2] Triton (triton-lang/triton), [3] ThunderKittens (HazyResearch/ThunderKittens), [4] Apache TVM, [5] Triton Gluon tutorial (01-intro.py), [6] NVIDIA CUTLASS CuTe DSL docs.
Sources (paraphrased): Horace He talk (from 34:00), Jane Street transcript; Matt Pharr, *The story of ispc: origins (part 1)* (quoting T. Foley). Video (from 34:00) | Talk transcript | Blog post

Minimal Tile level Access

Directly operating on lowest-level can be painful, as you likely noticed in the previous example.

How Users Think

HUMAN

Human reasoning is typically close to this style: use high-level operations over tensor slices (tiles), e.g.:

- copy
- GEMM
- add
- fill

Such subroutines can be reused.

AGENT

When agents handle instruction-level details directly, many errors are easy to introduce. Thinking in higher-level kernel algorithms is less error-prone, keeps programs concise, makes changes easier (context is always finite), and makes it easier to compose optimizations.

TIRx Mechanism

TIRx provides a first-class mechanism for this: deterministic operator dispatch.

1

Explicit layout on tensors

Layout (swizzle + tile) attached to each tensor determines logical-to-physical mapping, provide efficient access to certain tile-slice patterns.

2

Execution scopes

Execution scopes are explicit (kernel/CTA/warpgroup/warp/thread), defining exactly which thread group executes each operator.

3

Deterministic operator dispatch over tiles

Tile-level operators (Tx.copy, Tx.gemm_async, Tx.cast) on tensor tile slices, matching high-level reasoning. They are deterministically dispatched to low-level PTX based on layout, slice and execution scope.

TIRx: Determinstic Op Dispatch

TIRx: Deterministic Op Dispatch over Tiles

```
A_layout = Tx.ComposeLayout(
    Tx.SwizzleLayout(3, 3, 3, swizzle_inner=True),
    Tx.TileLayout(Tx.S[(128, 1, 64) : (64, 128 * 64, 1)]),
)
B_layout = Tx.ComposeLayout(
    Tx.SwizzleLayout(3, 3, 3, swizzle_inner=True),
    Tx.TileLayout(Tx.S[(128, 1, 64) : (64, 128 * 64, 1)]),
)
Dreg_layout = TileLayout(Tx.S[(128, 128) : (1@Tx.tid_in_wg, 1@Tx.TCol)])

@Tx.prim_func(tirx=True)
def gemm_fp16(A, B, D):
  with Tx.kernel():
    bx, by = Tx.cta_id([...], parent="kernel")
    wg_id = Tx.warpgroup_id([1], parent="cta")
    warp_id = Tx.warp_id([4], parent="warpgroup")
    lane_id = Tx.thread_id([32], parent="warp")

    pool = Tx.PoolAllocator()
    Asmem = pool.alloc((128,64), "float16", layout=A_layout)
    Bsmem = pool.alloc((128,64), "float16", layout=B_layout)
    pool.commit()
    Tx.ptx.tcgen05.alloc(tmem_addr, n_cols=512)

    with Tx.cta():              # all 128 threads
      Tx.copy(Asmem[:,:], A[:, :])
      Tx.copy(Bsmem[:,:], B[:, :])
    Tx.cuda.cta_sync()

    with Tx.thread()[Tx.elect_sync()]:  # 1 thread
      Tx.gemm_async(tmem[:,:], Asmem[:,:], Bsmem[:,:], accum=False, dispatch="tcgen05", cta_group=1)
      Tx.ptx.tcgen05.commit(mma_bar, cta_group=1)
    Tx.ptx.mbarrier.try_wait(mma_bar, phase)

    # write back
    Dreg_thr = Tx.alloc_local((128), "float32")
    Dreg_f16_thr = Tx.alloc_local((128), "float16")
    Dreg_wg = Dreg_thr.view(128, 128, layout=Dreg_layout)
    Dreg_f16_wg = Dreg_f16_thr.view(128, 128, layout=Dreg_layout)
    with Tx.warpgroup():       # 128 threads
      Tx.copy(Dreg_wg[:, :], tmem[:,:])
      Tx.cast(Dreg_f16_wg[:, :], Dreg_wg[:, :])
      Tx.copy(D[:, :], Dreg_f16_wg[:, :])
```

1. Explicit layout on tensors
2. Execution scopes
3. Deterministic operator dispatch over tiles

1. Explicit layout on tensors

Layout (swizzle + tile) attached to each tensor determines logical-to-physical mapping, provide efficient access to certain tile-slice patterns.

Outline

Step 1: Sync Load + MMA + Sync Store

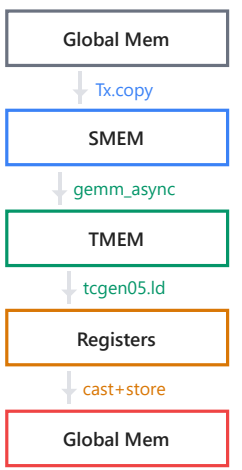
Step 1: Sync Load + MMA + Sync Store

The simplest working GEMM (128×128×64) — 0.1ms, 0.021 TFLOP/s

TIRX KERNEL (~40 LINES)

```
# Setup: SMEM + TMEM allocation
pool = Tx.PoolAllocator(buf.data)
Asmem = pool.alloc((128,64), "fp16", layout=A_layout)
Bsmem = pool.alloc((128,64), "fp16", layout=B_layout)
Tx.ptx.tcgen05.alloc(tmem_addr, n_cols=512)
Tx.ptx.mbarrier.init(mma_bar, 1)

# Load: all threads copy global → shared
with Tx.cta():
    Tx.copy(Asmem[:,:], A[m_st:m_st+128, k_st:k_st+64])
    Tx.copy(Bsmem[:,:], B[n_st:n_st+128, k_st:k_st+64])
Tx.cuda.cta_sync()

# Compute: single thread issues MMA
with Tx.thread()[Tx.ptx.elect_sync()]:
    Tx.gemm_async(tmem[:,:128], Asmem[:,:], Bsmem[:,:],
                  accum=k≠0, dispatch="tcgen05", cta_group=1)
    Tx.ptx.tcgen05.commit(mma_bar, cta_group=1)
Tx.ptx.mbarrier.try_wait(mma_bar, phase)

# Writeback: TMEM → registers → global
with Tx.warpgroup():
    Tx.copy(Dreg_wg[:,:], tmem[:,:128])    # tcgen05.ld
with Tx.thread():
    Tx.cast(Dreg_f16[:], Dreg[:])          # fp32 → fp16
    Tx.copy(D[m,n:n+128], Dreg_f16[:])     # store
```

DATAFLOW



Step 1: Full Code

Step 1: Sync GEMM — Full Kernel Code

Click a colored region to see its explanation · Single tile 128×128×64

```
with Tx.kernel():
    bx, by = Tx.cta_id([M // BLK_M, N // BLK_N], parent="kernel")
    wg_id = Tx.warpgroup_id([1], parent="cta")
    warp_id = Tx.warp_id([4], parent="warpgroup")
    lane_id = Tx.thread_id([32], parent="warp")

    # ── Shared memory allocation ──
    pool = Tx.PoolAllocator()
    tmem_addr = pool.alloc((1,), "uint32")
    mma_bar = pool.alloc((1,), "uint64", align=8)   # MMA done barrier
    Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
    Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
    pool.commit()

    # ── Init barrier + allocate TMEM ──
    if warp_id == 0:
        if lane_id == 0:
            Tx.ptx.mbarrier.init(mma_bar, 1)
        Tx.ptx.tcgen05.alloc(tmem_addr, n_cols=512, cta_group=1)
    Tx.ptx.fence.proxy_async("shared::cta")
    Tx.ptx.fence.mbarrier_init()
    Tx.cuda.cta_sync()

    tmem = Tx.decl_buffer((128, 512), "f32", scope="tmem")

    # ── Sync load: all threads copy GMEM → SMEM ──
    with Tx.cta():
        Tx.copy(Asmem[:,:], A[m_st:m_st+BLK_M, 0:BLK_K])
        Tx.copy(Bsmem[:,:], B[n_st:n_st+BLK_N, 0:BLK_K])
    Tx.cuda.cta_sync()
    Tx.ptx.tcgen05.fence.after_thread_sync()

    # ── MMA: one thread issues tcgen05.mma ──
    if warp_id == 0:
        with Tx.thread(parent="warp")[Tx.ptx.elect_sync()]:
            Tx.gemm_async(tmem[:,:BLK_N], Asmem[:,:], Bsmem[:,:],
                          accum=False, dispatch="tcgen05", cta_group=1)
            Tx.ptx.tcgen05.commit(mma_bar, cta_group=1)
```

CODE REGIONS

- Setup & Init
- Sync Load
- MMA Compute
- Writeback

Overview

The simplest GEMM: one 128×128×64 tile, fully synchronous. All 128 threads sync-load A,B from GMEM→SMEM, one elected thread issues MMA, all threads writeback.

> This is the skeleton — every later step builds on this structure.

Outline

Step 4: TMA Async Load

Step 4: TMA Async Load

Key change: all-thread sync copy → single-thread TMA dispatch with barrier tracking

Step 3 — Sync Load (all 128 threads)

```
# Every thread copies one row
with Tx.cta():
    Tx.copy(Asmem[:,:], A[m:m+128, k:k+64])
    Tx.copy(Bsmem[:,:], B[n:n+128, k:k+64])
Tx.cuda.cta_sync()

# All threads blocked until copy done
# Then MMA proceeds ...
Tx.ptx.tcgen05.fence.after_thread_sync()
Tx.gemm_async(tmem, Asmem, Bsmem, ...)
Tx.ptx.tcgen05.commit(mma_bar, ...)
Tx.ptx.mbarrier.try_wait(mma_bar, phase)

# Problem: 128 threads stalled on memory
# → 99%+ time in data movement
```

Step 4 — TMA Async (single thread)

```
# One thread dispatches HW DMA engine
@Tx.inline
def tma_load(k_st):
    tma_cfg = {"dispatch": "tma", "cta_group": 1,
               "mbar": tma_bar.ptr_to([0])}
    Tx.copy_async(Asmem[:,:], A[m_st:m_st+128, k_st:k_st+64], **tma_cfg)
    Tx.copy_async(Bsmem[:,:], B[n_st:n_st+128, k_st:k_st+64], **tma_cfg)
    Tx.ptx.mbarrier.arrive.expect_tx(tma_bar.ptr_to([0]), byte_count)

@Tx.inline
def mma(accum):
    Tx.ptx.mbarrier.try_wait(tma_bar.ptr_to([0]), phase_tma)
    phase_tma = phase_tma ^ 1
    Tx.ptx.tcgen05.fence.after_thread_sync()
    Tx.gemm_async(tmem[:,:128], Asmem[:,:], Bsmem[:,:],
                  accum=accum, dispatch="tcgen05", cta_group=1)
    Tx.ptx.tcgen05.commit(mma_bar.ptr_to([0]), cta_group=1)
```

HW DMA `copy_async(dispatch="tma")` offloads to the TMA engine — 127 threads freed, memory bus saturated

BARRIER SYNC `arrive.expect_tx(bytes)` tells barrier how many bytes TMA will write. `try_wait(phase)` blocks until done. Phase flips each iteration.

SINGLE THREAD `tid == 0` issues both TMA load and MMA sequentially — no thread divergence overhead

Step 4: TMA Async Store (Writeback)

Step 4: TMA Async Store (Writeback)

TMEM cannot write to GMEM directly — data flows through Registers and SMEM

WRITEBACK DATA PATH

Phase 1: TMEM → Registers (chunked 8-col)

```
# Allocate full-row fp16 buffer for 128 cols
Dreg_16b = Tx.alloc_local((128,), "float16")
# Loop 16 times (128 / 8 = 16)
for no in Tx.unroll(128 // 8):
    # Allocate 8-col fp32 temp register
    Dreg = Tx.alloc_local((8,), "float32")
    # Warpgroup collective: 128 threads read 8 cols from TMEM
    with Tx.warpgroup():
        Dreg_wg = Dreg.view(128, 8, layout= ... )
        Tx.copy(Dreg_wg[:,:], tmem[:, no*8:no*8+8])
    # Per-thread cast: fp32 → fp16
    with Tx.thread():
        Tx.cast(Dreg_16b[no*8:(no+1)*8], Dreg[:])
```

Phase 2: Registers → SMEM → TMA Store

```
# Loop 2 times (128 / 64 = 2)
for no in Tx.unroll(128 // 64):
    # Each thread writes its 64-col slice to SMEM
    with Tx.thread():
        Tx.copy(Dsmem[warp_id*32+lane_id, :],
                Dreg_16b[no*64:(no+1)*64])
        # Make SMEM writes visible to TMA engine
        Tx.ptx.fence.proxy_async("shared::cta")
    # Wait all threads done writing SMEM
    Tx.cuda.warpgroup_sync(10)
    # Elect 1 thread to issue TMA store: SMEM → GMEM
    with Tx.thread(parent="warpgroup")[Tx.ptx.elect_sync()]:
        n_st_epi = Tx.meta_var(n_st + no * 64)
        Tx.copy_async(D[m_st:m_st+128, n_st_epi:n_st_epi+64],
                      Dsmem[:,:], dispatch="tma")
        # Commit and wait for TMA store completion
        Tx.ptx.cp_async.bulk.commit_group()
```

WHY 8-COL CHUNKS? `tcgen05.ld` reads max 8 cols per warpgroup copy. Loop `128/8=16` times for full row.

WHY GO THROUGH SMEM? TMA only accesses **shared memory**. Path: TMEM→Reg→cast fp16→SMEM→TMA store→GMEM.

EPI_N=64 TILING `Dsmem` is 128×64, not 128×128 — halves SMEM. Loop `128/64=2` times, each TMA-storing a 128×64 tile.

Step 4: Full Code

Step 4: TMA Async — Full Kernel Code

Click a colored region to see its explanation

```
with Tx.kernel():
    bx, by = Tx.cta_id([M // BLK_M, N // BLK_N], parent="kernel")
    wg_id = Tx.warpgroup_id([1], parent="cta")
    warp_id = Tx.warp_id([4], parent="warpgroup")
    lane_id = Tx.thread_id([32], parent="warp")

    # ── Shared memory allocation ──
    pool = Tx.PoolAllocator()
    tmem_addr = pool.alloc((1,), "uint32")
    tma_bar = pool.alloc((1,), "uint64", align=8)   # TMA done → MMA can start
    mma_bar = pool.alloc((1,), "uint64", align=8)   # MMA done → safe to read TMEM
    Asmem = pool.alloc((BLK_M, BLK_K), a_type, layout=A_layout)
    Bsmem = pool.alloc((BLK_N, BLK_K), b_type, layout=B_layout)
    Dsmem = pool.alloc((BLK_M, EPI_N), d_type, layout=D_layout)
    pool.commit()

    # ── Init barriers + allocate TMEM ──
    Tx.ptx.mbarrier.init(mma_bar, 1)
    Tx.ptx.mbarrier.init(tma_bar, 1)
    Tx.ptx.tcgen05.alloc(tmem_addr, n_cols=512, cta_group=1)
    Tx.cuda.cta_sync()

    tmem = Tx.decl_buffer((128, 512), "f32", scope="tmem")
    phase_tma = 0
```

CODE REGIONS

- Setup & Init
- TMA Async Load
- MMA Compute
- Main Loop
- Writeback

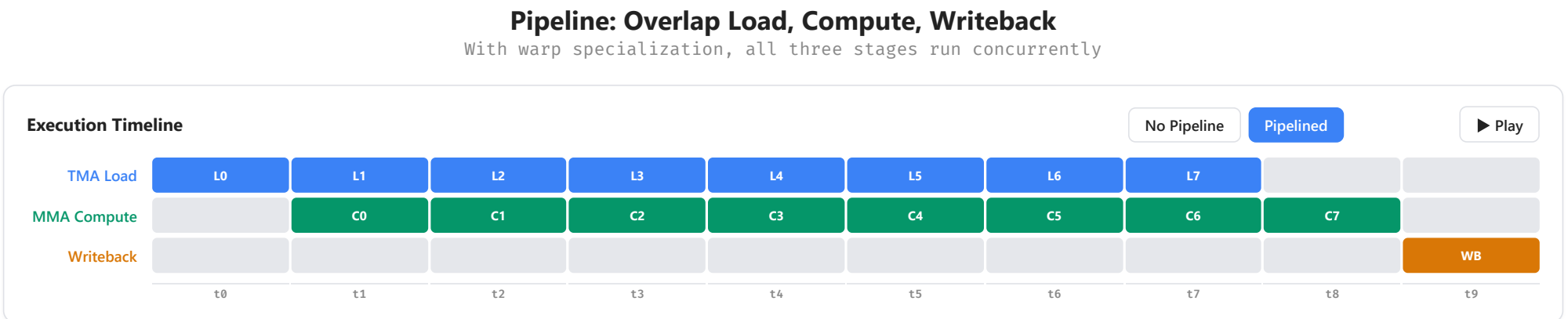
Overview

Step 4 uses **TMA** (Tensor Memory Accelerator) for async global→SMEM loads, replacing sync `Tx.copy`. A **single elected thread** issues all TMA loads and MMA instructions. Two barriers coordinate: `tma_bar` (load→compute) and `mma_bar` (compute done).

> Key change from Step 1: `Tx.copy` → `Tx.copy_async(dispatch="tma")`

Outline

Pipeline: Overlap Load, Compute, Writeback



Why pipeline? Without pipelining, each tile waits for Load → Compute → Store to finish before starting the next. With 3+ pipeline stages, TMA loads tile *i+2* while MMA computes tile *i+1* while Writeback stores tile *i* — hiding latency and saturating all hardware units simultaneously.

Warp Specialization: Overlap Everything

Barrier Roles

`tma2mma[PIPE_DEPTH]` ●→●
TMA signals MMA: SMEM data ready for stage k
`Type: TMABar (TMA arrive with byte count)`

`mma2tma[PIPE_DEPTH]` ●→●
MMA signals TMA: SMEM buffer free, can reuse stage k
`Type: TCGen05Bar (tcgen05.commit)`

`mma2ld` ●→●
MMA signals Writeback: TMEM accumulation complete
`Type: TCGen05Bar (tcgen05.commit)`

`ld2mma` ●→●
Writeback signals MMA: TMEM read complete, safe to overwrite
`Type: MBarrier (128 threads arrive)`

Step 7: Warp Specialization — Code

Step 7: Warp Specialization — Code

Three actors with typed barriers: TMABar, TCGen05Bar, MBarrier, PipelineState

```
tma2mma = TMABar(pool, PIPE_DEPTH, "tma2mma")   → # TMA → MMA
mma2tma = TCGen05Bar(pool, PIPE_DEPTH, "mma2tma")   → # MMA → TMA
mma2ld  = TCGen05Bar(pool, 1, "mma2ld")   → # MMA → Writeback
ld2mma  = MBarrier(pool, 1, "ld2mma")   → # Writeback → MMA
```

TMA Producer (WG1/warp3)

```
if wg_id == 1 and warp_id == 3:
    tma_phase = PipelineState("tma", PIPE_DEPTH)
    tma_phase.init(is_producer=True)

    @Tx.inline
    def tma_load_stage(stage, k_st):
        mma2tma.wait(stage, tma_phase.phase)
        tma_cfg = {"dispatch": "tma", "cta_group": 1,
                   "mbar": tma2mma.ptr_to([stage])}
        Tx.copy_async(Asmem[stage,:,:], A[ ... ], **tma_cfg)
        Tx.copy_async(Bsmem[stage,:,:], B[ ... ], **tma_cfg)
        tma2mma.arrive(stage, byte_count)

    @Tx.inline
    def tma_load():
        for k in Tx.serial(K_TILES):
            tma_load_stage(tma_phase.stage, k*BLK_K)
            tma_phase.move_to_next_stage()

    with Tx.thread(parent="warp")[Tx.ptx.elect_sync()]:
```

MMA Consumer (WG1/warp0)

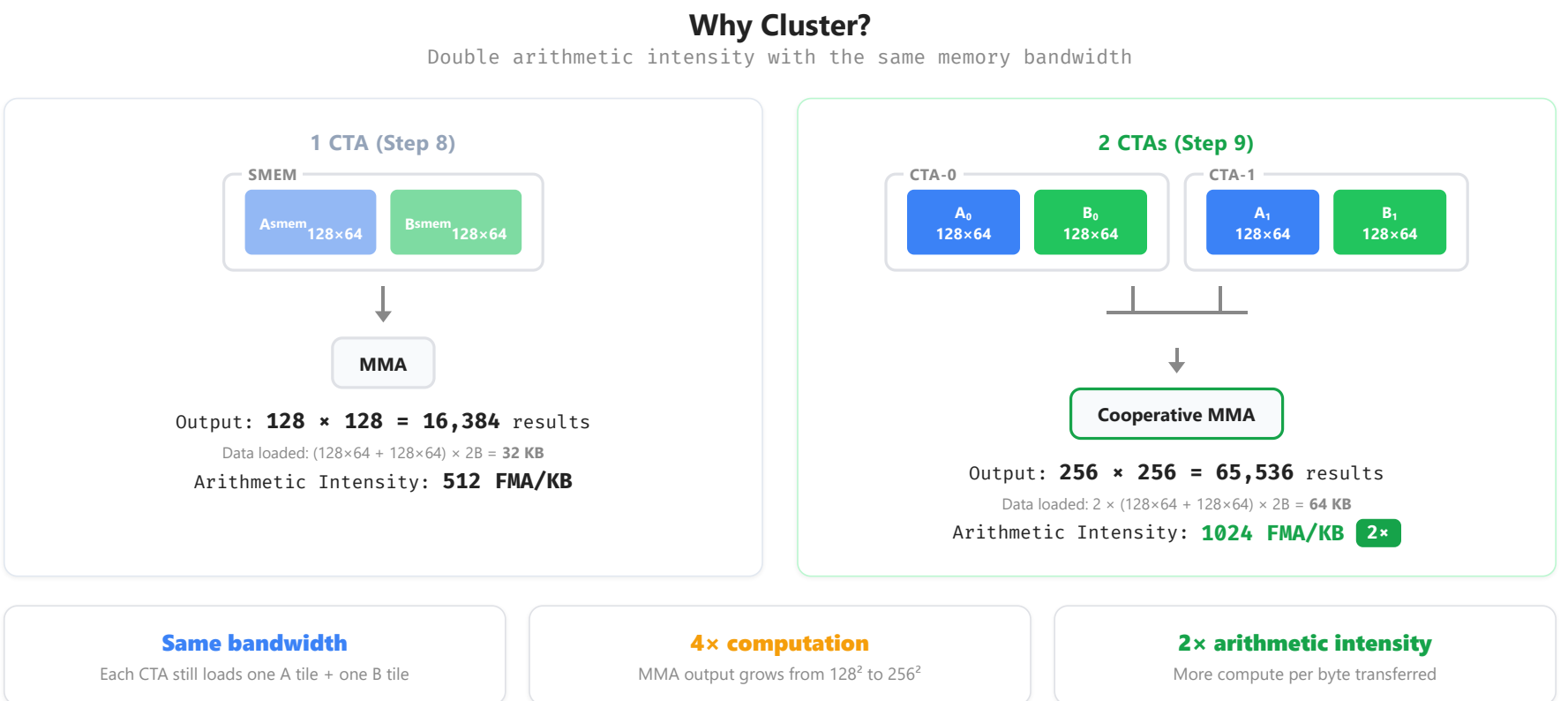
```
elif warp_id == 0:
    mma_phase = PipelineState("mma", PIPE_DEPTH)
    mma_phase.init(is_producer=False)
    ld_phase = PipelineState("ld", 1)
    ld_phase.init(is_producer=True)

    @Tx.inline
    def mma_stage(stage):
        tma2mma.wait(stage, mma_phase.phase)
        Tx.gemm_async(tmem[:,:BLK_N],
            Asmem[stage,:,:], Bsmem[stage,:,:],
            accum=accum, dispatch="tcgen05",
            cta_group=1)
        accum = 1
        mma2tma.arrive(stage, cta_group=1, cta_mask=1)

    @Tx.inline
    def mma():
        ld2mma.wait(0, ld_phase.phase)
        ld_phase.move_to_next_stage()
```

Writeback (WG0)

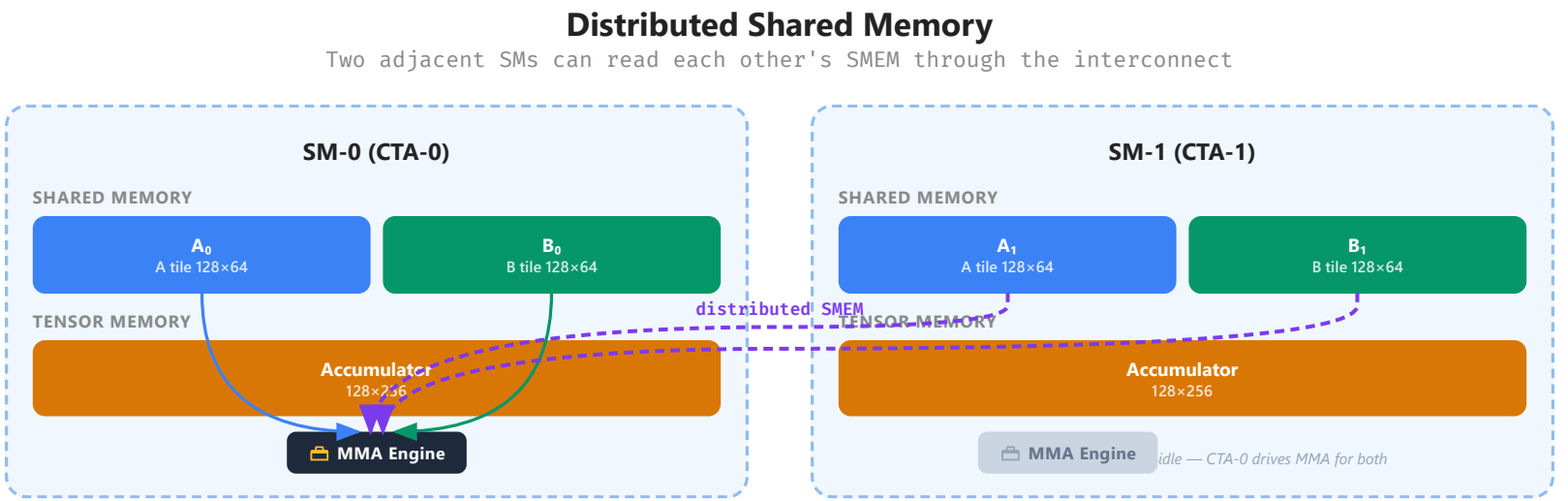
```
elif wg_id == 0:
    wb_phase = PipelineState("wb", 1)
    wb_phase.init(is_producer=False)

    @Tx.inline
    def writeback():
        mma2ld.wait(0, wb_phase.phase)
        wb_phase.move_to_next_stage()
        Tx.ptx.tcgen05.fence.after_thread_sync()

        # TMEM → registers (chunked)
        for no in Tx.unroll(MMA_N // TMEM_LD_N):
            Dreg = Tx.alloc_local((TMEM_LD_N,), "f32")
            with Tx.warpgroup():
                Tx.copy(Dreg_wg, tmem[:,no*8:no*8+8])
            with Tx.thread():
                Tx.cast(Dreg_16b[no*8:(no+1)*8], Dreg)

        ld2mma.arrive(0, cta_id=0, pred=True)
```

Outline

Why Cluster?


Why Cluster?
Double arithmetic intensity with the same memory bandwidth
1 CTA (Step 8)
SMEM
Asmem 128×64
Bsmem 128×64
MMA
Output: 128 × 128 = 16,384 results
Data loaded: (128×64 + 128×64) × 2B = 32 KB
Arithmetic Intensity: 512 FMA/KB
2 CTAs (Step 9)
CTA-0
A0 128×64
B0 128×64
CTA-1
A1 128×64
B1 128×64
Cooperative MMA
Output: 256 × 256 = 65,536 results
Data loaded: 2 × (128×64 + 128×64) × 2B = 64 KB
Arithmetic Intensity: 1024 FMA/KB
2×
Same bandwidth
Each CTA still loads one A tile + one B tile
4× computation
MMA output grows from 128² to 256²
2× arithmetic intensity
More compute per byte transferred

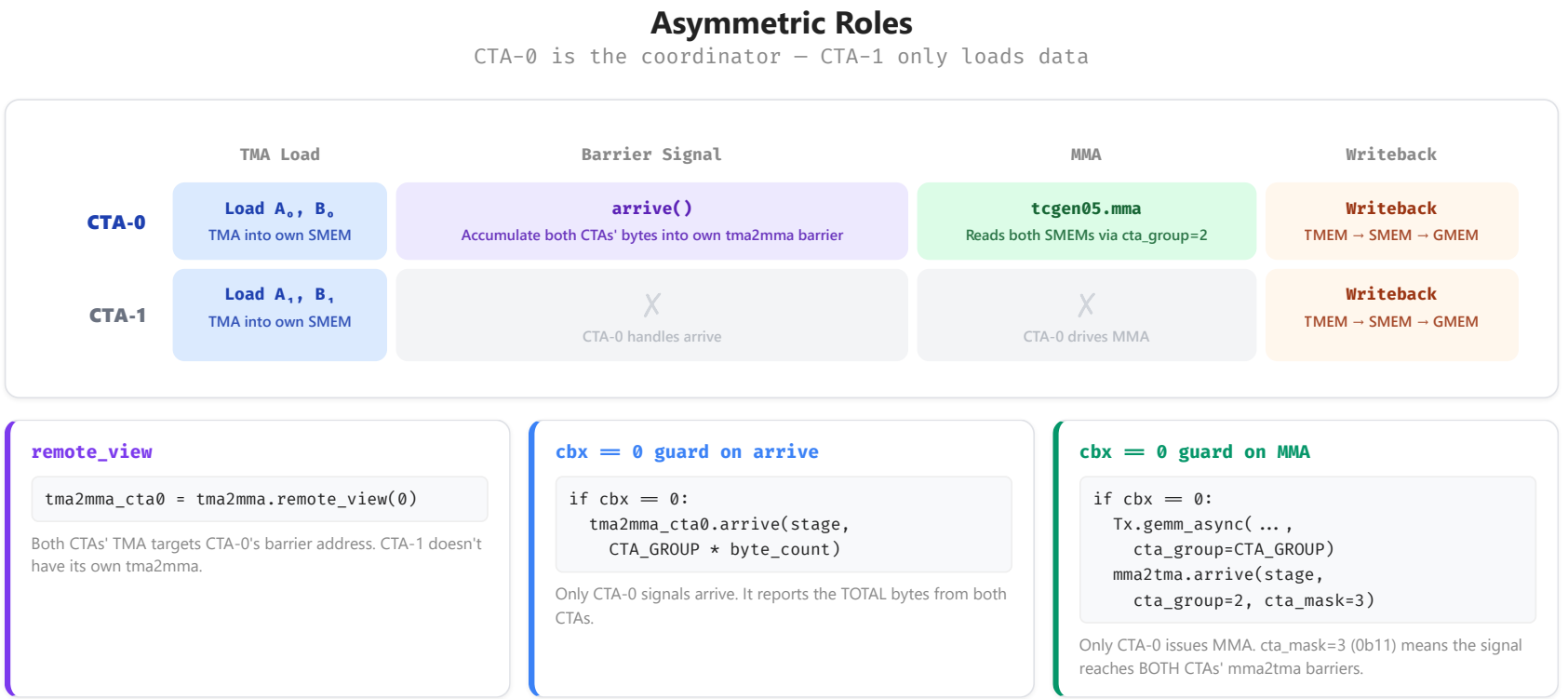
Distributed Shared Memory



`tcgen05.mma` with **cta_group=2** reads SMEM from **both** CTAs in the cluster

Result is distributed: 128 rows in CTA-0's TMEM + 128 rows in CTA-1's TMEM → **256×256 total**

Asymmetric Roles: CTA-0 vs CTA-1



remote_view

```
tma2mma_cta0 = tma2mma.remote_view(0)
```

Both CTAs' TMA targets CTA-0's barrier address. CTA-1 doesn't have its own tma2mma.

cbx == 0 guard on arrive

```
if cbx == 0:
  tma2mma_cta0.arrive(stage,
    CTA_GROUP * byte_count)
```

Only CTA-0 signals arrive. It reports the TOTAL bytes from both CTAs.

cbx == 0 guard on MMA

```
if cbx == 0:
  Tx.gemm_async( ... ,
    cta_group=CTA_GROUP)
  mma2tma.arrive(stage,
    cta_group=2, cta_mask=3)
```

Only CTA-0 issues MMA. cta_mask=3 (0b11) means the signal reaches BOTH CTAs' mma2tma barriers.

Step 9: 2-CTA Cluster — Code

Step 9: 2-CTA Cluster — Code

Same three-role structure as Step 7 — only highlighted lines change

```
CTA_GROUP = 2                                  # NEW
tma2mma = TMABar(pool, PIPE_DEPTH, "tma2mma")  → # TMA → MMA
mma2tma = TCGen05Bar(pool, PIPE_DEPTH, "mma2tma")  → # MMA → TMA
mma2ld  = TCGen05Bar(pool, 1, "mma2ld")  → # MMA → Writeback
ld2mma  = MBarrier(pool, 1, "ld2mma")  → # Writeback → MMA
tma2mma_cta0 = tma2mma.remote_view(0)          # NEW: cross-CTA barrier view
ld2mma.init(128 * CTA_GROUP)                   # CHANGED: was 128
```

TMA Producer (WG1/warp3)

```
if wg_id == 1 and warp_id == 3:
    tma_phase = PipelineState("tma", PIPE_DEPTH)
    tma_phase.init(is_producer=True)

    @Tx.inline
    def tma_load_stage(stage, k_st):
        mma2tma.wait(stage, tma_phase.phase)
        tma_cfg = {"dispatch": "tma", "cta_group": CTA_GROUI
                   "mbar": tma2mma_cta0.ptr_to([stage])}
        Tx.copy_async(Asmem[stage,:,:], A[ ... ], **tma_cfg)
        Tx.copy_async(Bsmem[stage,:,:], B[ ... ], **tma_cfg)
        if cbx == 0:
            tma2mma_cta0.arrive(stage,
                CTA_GROUP * byte_count)
```

MMA Consumer (WG1/warp0)

```
elif warp_id == 0:
    if cbx == 0:
        mma_phase = PipelineState("mma", PIPE_DEPTH)
        mma_phase.init(is_producer=False)
        ld_phase = PipelineState("ld", 1)
        ld_phase.init(is_producer=True)

        @Tx.inline
        def mma_stage(stage):
            tma2mma.wait(stage, mma_phase.phase)
            Tx.gemm_async(tmem[:,:MMA_N],
                Asmem[stage,:,:], Bsmem[stage,:,:],
                accum=accum, dispatch="tcgen05",
                  cta_group=CTA_GROUP)
            accum = 1
            mma2tma.arrive(stage
```

Writeback (WG0)

```
elif wg_id == 0:
    wb_phase = PipelineState("wb", 1)
    wb_phase.init(is_producer=False)

    @Tx.inline
    def writeback():
        mma2ld.wait(0, wb_phase.phase)
        wb_phase.move_to_next_stage()
        Tx.ptx.tcgen05.fence.after_thread_sync()

        # TMEM → registers (chunked)
        for no in Tx.unroll(MMA_N // TMEM_LD_N):
            Dreg = Tx.alloc_local((TMEM_LD_N,), "f32")
            with Tx.warpgroup():
```

Key changes: CTA_GROUP=2 · remote_view(0) for cross-CTA barrier · cbx==0 guards · cta_mask=1→3 · cluster_sync

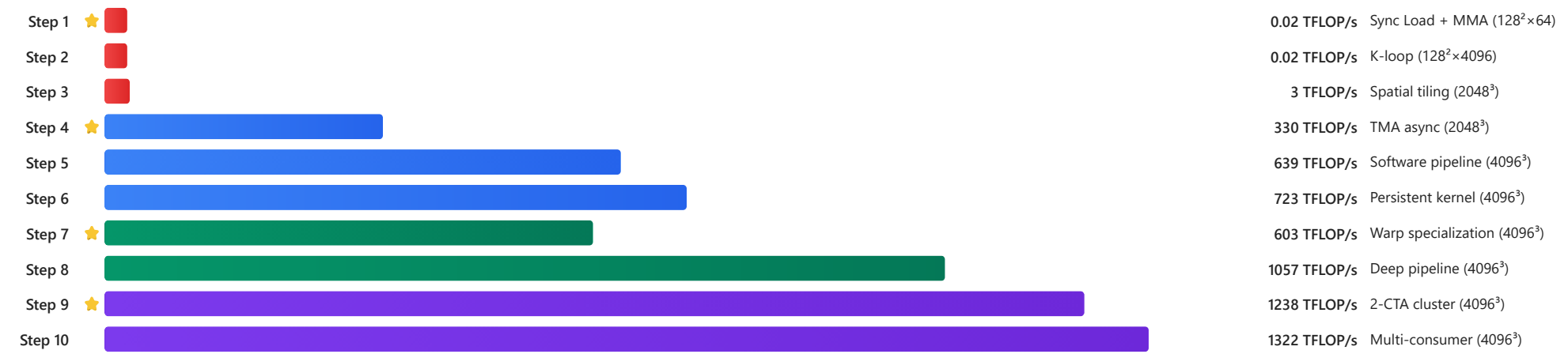
Outline

The Optimization Journey

The Optimization Journey

From 0.02 to 1322 TFLOP/s in 10 steps of TIRx

PERFORMANCE JOURNEY



TMA: Hardware Data Mover

Let hardware do the work, free 127 threads

tcgen05.mma: Tensor Memory

TMEM avoids register pressure, single-thread dispatch

Warp Specialization

Dedicated roles: load / compute / writeback overlap

2SM Cluster

2× compute per SMEM tile, match cuBLAS

Next: Assignment — Build your own GEMM kernel step by step (10 steps, incremental testing)